EVO

The boundary layer for AI agents

A build plan for one day, from zero, with the clock running.

EVENT	Ship Happens · AWS Student Builder Group × ADIGPS
DATE	Saturday 31 October 2026 · 9:00 - 18:30 IST
VENUE	Juspay Technologies · Gurugram
FREEZE	4:00 PM · demos 4:30 · awards 6:00

Everyone in that room will build an agent that does something.
You are building the thing the agents have to ask.

That is the only category of one available today.

Every command, prompt and file path in this document has been run against the repository at github.com/BHAVIT-EVO/architecture-v1.
Where something is a judgement call rather than a fact, it says so.

PART ONE

What you are actually entering

A hackathon is not a build contest. It is a compression contest: how much credible uniqueness can you put in front of a tired judge in three minutes. You already have more substance than most teams will produce today — a deterministic engine, a real corpus, a verifier that fails the build when it is wrong. What you do not yet have is a product a stranger can touch.

So today is not about writing more engine. It is about turning what exists into three surfaces a judge can operate without you talking:

- A service. Something running on a port, with endpoints. Not a page.
- An interface any agent can call. That is the new thing — it did not exist in the repository this morning.
- A voice. Ask in Hindi, hear the answer in Hindi, and watch the boundary resolve underneath it.

Three things a judge decides in thirty seconds

	The question in their head	What answers it
1	Do I understand this?	One sentence, repeated three times, never varied.
2	Is it real?	Real events, real numbers, and a failure you cause on purpose.
3	Is it theirs?	A refusal, a gate, a language switch. Things you cannot fake live.

> **THE UNFAIR ADVANTAGE**
> Every other team will demo an agent that does a task. When they are asked "how does it know what to work on?", they will say "the user tells it" or "we pass the folder". You are the answer to that question. Do not compete on what the agent does. Compete on what it is allowed to touch.

What wins, specifically

Judges at a student hackathon reward four things, in this order: a demo that does not break, a problem they recognise, evidence that it is not a wrapper, and a story they can repeat to someone else. Notice that novelty is third. You do not need to be the most original project in the room. You need to be the one nobody can place in an existing category.

Judging pressure	How Evo answers it
"Is this another OpenAI wrapper?"	The decision path has no model in it. Turn off the wifi and it still works.
"Isn't this just a folder?"	A folder is declared in advance. This is derived from behaviour and carries its own evidence.
"How do I know it's right?"	It refuses more than it asserts, and it shows what it rejected.
"Why now?"	Agents just started acting on real machines. They need jurisdiction before they can be trusted with one.

The single most memorable thing you will do today is this: let a judge try to merge two bodies of work, refuse them, then let them declare it anyway, and watch the build fail. People remember the product that told them no.

PART TWO

The demo product and the real product

These are the same thing at two distances. Say it that way and the ambition lands without sounding like a pitch deck.

	Demo product (today)	Real product (what you say next)
What it is	A local service on one laptop. Any program on the machine can ask which body of work a resource belongs to.	The same service shipped inside the desktop app. Every agent on the machine queries it as infrastructure.
Who uses it	You, in front of a judge, in three minutes.	Knowledge workers whose agents act on their machines; the agents themselves.
How it earns	Nothing. It is a demonstration.	Per seat, prosumer first, then team. The boundary is the moat, not the model.
Why it is defensible	Not defensible. It is a demo.	The corpus and the verifier: a competitor can copy the API in a week and cannot copy the ten defects you found by measuring.

The four surfaces you will show

Know which is which, because a judge will ask and a wrong answer costs you the room.

#	Surface	What the judge sees	Built
1	Service	curl on a terminal: works, resume bundles, a refusal. JSON, not a screenshot.	Today
2	MCP	An agent asking Evo where it is, and getting an answer with evidence.	Today — new
3	Voice	A question in Hindi, an answer in Hindi, the boundary resolving underneath.	Today
4	Engine	A gate that fails the build when two works fuse. The reason to believe the rest.	Months

> **WHAT IS GENUINELY NEW TODAY**
> The repository has had no MCP server and no socket listener. Today you give Evo a mouth: an HTTP surface, a protocol any agent can speak, and a voice. That is a real line to say out loud - "this morning Evo was a library; tonight it is addressable" - and it is true, which is the point.

The one-sentence version, memorised

Every agent today is told where its work begins and ends. Evo derives it, locally, and lets any agent ask.

That is the whole pitch. Everything else in the demo is evidence for that sentence. If you lose your place, say it again.

PART THREE

How it reads Evo

This is the part you must be able to draw on a napkin, because the question "how does the new bit read the engine?" is the one that separates you from someone who wired an API to a mock.

```
sample_events.json      10,392 recorded events
      |
      |   cargo run -p evo-threads --bin boundary_dump
      v
evo-threads ENGINE     Rust, deterministic, offline, no HTTP client
      |
      |   Engine::briefs()  ->  Vec<WorkBrief>
      v
threads.json           the one contract between the old and the new
      |
      |   python3 evo_server.py     (one process, two mouths)
      v
evo_server.py
      |
      +--- serve  :8765   ->  evo_ui.html  ->  Sarvam voice
      |
      +--- mcp    stdio   ->  any agent   ->  list / find / resume
```

Why each layer exists, in one line each

Layer	Its job	If you removed it
Engine	Decides the boundary. Deterministic, offline, no model.	There is no product, only a UI.
threads.json	The contract. One file, one shape, no opinions.	The service would have to link Rust to be useful.
Service	Makes the boundary addressable: HTTP, JSON, on a port.	Nothing else can use it. It stays a demo of a demo.
MCP	Makes it speakable by agents, not just by your own UI.	You are the only client. That is a toy.
Voice	Makes the answer reachable without typing, in your language.	One less sense; still a product.

The shape of threads.json

This is what the engine hands over. Read it once and you can answer any technical question about the demo.

```
{
  "source": "engine",
  "events": 10392,
  "works": [
    {
      "id": 3,
      "name": "Unisex Hiking & Trekking Clothes ...",   <- strongest anchor
      "status": "Active",
      "episodes": 191,                                  <- sittings, not pages
      "anchors": ["arcteryx...", "hats-and-caps..."],    <- what you made / returned to
      "companions": ["google", "chrome"],               <- context, decays
      "contested": [],                                  <- shared identity, shown not hidden
      "resume_point": "https://arcteryx.com/...",
      "resume_reason": "Last seen: https://arcteryx.com/...",
      "restore_set": ["...", "..."],                    <- the neighborhood, not the museum
      "fork": null                                      <- retained ambiguity
    }
  ]
}
```

> **IF A JUDGE ASKS WHERE THE AI IS**
> "Deliberately not in the decision. The boundary has to be deterministic and auditable or it is not a boundary - it is a suggestion. AI is a consumer of this, not the author of it. That is why the answer is the same twice, and why I can show you a test that fails the build when it gets it wrong."

PART FOUR

What already exists, so you do not rebuild it

Everything below is committed and pushed. Your job is to run it, fix what breaks on your machine, and add the two things that do not exist yet: the voice path and the protocol probe you will demo with.

File	What it does	Status
crates/evo-threads/src/bin/boundary_dump.rs	Replays the corpus through the engine and writes threads.json.	Written, compiles against the verified API, not yet run
exports/ship-happens/evo_server.py	HTTP service on :8765 and an MCP server over stdio. Stdlib only.	Tested: /works, /ask, /health, MCP handshake
exports/ship-happens/evo_ui.html	The product surface. Works rail, resume bundle, ask bar, language toggle.	Tested against the service
exports/ship-happens/boundary.html	The offline explorer, with the two break-it buttons and the verifier.	Tested, works with no server
exports/ship-happens/boundary.json	Label-derived fallback if the Rust build fails.	Present

The file map, so you always know where you are

```
architecture-v1/
  crates/evo-threads/src/          the engine (do not edit today)
    bin/boundary_dump.rs           events -> threads.json      [your bridge]
    bin/regression.rs              the gate that exits 1 on fusion
  exports/ship-happens/            everything you demo
    evo_server.py                  service + MCP               [demo core]
    evo_ui.html                    product UI
    boundary.html                  offline explorer
    threads.json                   engine output (produced today)
    boundary.json                  label fallback
    mcp_probe.py                   the agent you demo with     [you write]
    RUNBOOK.md                     this plan, in markdown
```

> **DO NOT TOUCH THE ENGINE TODAY**
> Not one line. It is the thing you are claiming is real, and a rushed edit at 12:40 that does not compile costs you the afternoon. If you find a bug, write it on a page and fix it on Monday.

PART FIVE

Sprint 01: 10:10 to 14:00

Three hours and fifty minutes. Five blocks. Each block ends with something you can show, so that if you stop at any point you still have a demo. That property matters more than finishing.

Block	Time	You produce	If it fails
A	10:10 - 10:40	Kiro open, engine verified, threads.json written.	Use boundary.json. Say the engine output is precomputed.
B	10:40 - 11:40	Service on :8765, UI loading real works.	Fall back to boundary.html, which needs no server.
C	11:40 - 12:40	An agent asking Evo and getting evidence back.	Drop MCP. Demo the HTTP endpoint with curl.
D	12:40 - 13:40	Voice in Hindi, and a recorded backup.	Drop voice. Never drop the backup video.
E	13:40 - 14:00	Frozen, one full run-through, committed.	-

The rule that keeps you on schedule

Each block has a twenty-minute overrun limit. If you are past it, take the failure path, say the fallback line once, and move on. A demo missing its third act still wins. A demo that is still compiling at 4:10 cannot be submitted.

10:10 - 10:40

Block A - Setup and the engine proof

Goal: by 10:40 you have either a threads.json from the real engine, or a written decision to use the fallback. No ambiguity, no half-built Rust.

A1 · Ten minutes, in this order

```
# 1. open the repo
cd architecture-v1

# 2. prove the engine runs at all (5-8 min the first time)
cargo run -p evo-threads --bin regression -- \
  --events tools/threads-regression/sample_events.json \
  --labels tools/threads-regression/ground_truth_labels.json

# 3. the bridge: write the engine's own boundary to JSON
cargo run -p evo-threads --bin boundary_dump -- \
  --events tools/threads-regression/sample_events.json \
  --out exports/ship-happens/threads.json

# 4. look at it
python3 -c "import json;d=json.load(open('exports/ship-happens/threads.json'));\
print(d['source'],len(d['works']),'works');\
[print(w['id'],w['episodes'],w['name'][:50]) for w in d['works'][:8]]"
```

The regression binary should print a report and exit 0. If it exits 1, that is the gate working, and it is worth thirty seconds of your time to understand why before you move on.

A2 · Kiro, in the same ten minutes

Sign in with AWS Builder ID, open the repo folder, and write the steering file. Kiro reads it on every request; without it the agent will try to add pip dependencies and an HTTP client to your deterministic engine.

PASTE INTO KIRO · .kiro/steering/product.md

```
Create a steering file at .kiro/steering/product.md with exactly this
content. Do not add to it.

# Evo

Evo derives a person's work boundary locally and exposes it for other
programs to query. A "work" is a body of related activity on one machine.

Hard rules. Never violate these; if a task seems to require it, stop and
say so instead:

1. The boundary is computed locally by deterministic rules. No model
   decides it. The same input must produce the same output twice.
2. No hard merges of stored state. Two works fuse only on an explicit
   human declaration. No inference path may reach a merge.
3. Split is free. Merge is never free and never automatic.
4. Any external API (Sarvam, a model, a cloud) may CONSUME the boundary.
   Nothing external may INFLUENCE how the boundary is computed.
5. Python: standard library only. No pip installs, no frameworks.
   Every external call degrades gracefully to a local answer.
6. Never edit crates/. The engine is frozen for this event.

Then show me the file and confirm it exists.
```

A3 · The one thing you must be able to explain

PASTE INTO KIRO · read-only

```
Read crates/evo-threads/src/lib.rs and src/brief.rs. Do not write code.
Explain in under 250 words, in plain English: what a work is, what a
brief contains, how resume_point and restore_set are chosen, and why the
engine has no concept of a filename or an application name.

Then give me three questions a judge might ask about this design and the
one-sentence answer to each.
```

> **WHY THIS PROMPT IS NOT OPTIONAL**
> You will be asked "how does it decide?". If your answer is "the AI figures it out", you have just become every other project in the room. If your answer is "attention is serial, work is not, so it segments episodes and joins them by what you made rather than where you were", you are the only person in the building who can say that.

10:40 - 11:40

Block B - The service

Goal: something running on a port that a stranger could curl. This is the difference between a project and a page.

B1 · Run it before you change it

```
cd exports/ship-happens

# engine output if you have it, labels otherwise - it picks whichever exists
python3 evo_server.py serve

# in another terminal
curl -s localhost:8765/health
curl -s localhost:8765/works | head -c 400
curl -s "localhost:8765/ask?q=where+did+i+look+for+hiking+clothes"
curl -s "localhost:8765/ask?q=capital+of+france"
```

You want four answers: healthy, a list of works, "gear" with evidence, and a refusal. If the refusal is missing, your boundary is guessing and the whole thesis is broken. Fix that first, before anything cosmetic.

B2 · Then make it yours

PASTE INTO KIRO · exports/ship-happens/evo_server.py

```
Read exports/ship-happens/evo_server.py. Do not rewrite it.

Add one endpoint: GET /evidence?q=<question>

It must return the same answer as /ask, plus, for the winning work, the
three strongest individual pieces of evidence with their weights, in the
order the scorer produced them - not re-sorted, not summarised.

Constraints: standard library only, keep the existing response shape for
/ask unchanged, and do not touch load_boundary() or normalise().
```

B3 · The line you say while it runs

Open the UI in a browser. Say this, and only this, the first time:

> **THE SERVICE LINE**
> "That is not a screenshot. That is a service on this laptop, reading a file the Rust engine wrote twenty minutes ago. There is no cloud in this, which is why it still works if I turn off the wifi - and I will, later."

Then move on. Do not tour the code. The code is not the demo; the refusal is.

11:40 - 12:40

Block C - The protocol, and the agent you demo with

This is the new thing. Until today, nothing outside the repository could ask Evo anything. MCP is how an agent asks.

C1 · Confirm the server speaks

```
printf '%s\n' \
 '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{}}' \
 '{"jsonrpc":"2.0","id":2,"method":"tools/list"}' \
  '{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"find_work","arguments":{"query":"arcteryx  for
the trek"}}}' \
  | python3 evo_server.py mcp
```

Three JSON responses. The third says trek, shows the evidence, and names the work it rejected. That is your entire pitch, in a protocol message.

C2 · Write the probe — this is your demo agent

You are not going to rely on Claude Desktop connecting on venue wifi. You will have your own client, twenty lines long, that visibly performs the handshake and prints the exchange. It is more convincing than a chat window because the judge can see the protocol.

PASTE INTO KIRO · exports/ship-happens/mcp_probe.py (new file)

```
Write exports/ship-happens/mcp_probe.py. Standard library only.

It takes a question from argv[1] and talks to `evo_server.py mcp` over
stdin/stdout as a real MCP client:

1. send initialize, read the result, print the server name
2. send notifications/initialized
3. send tools/list, print the tool names
4. send tools/call with name "find_work" and the question
5. print the text content of the response, indented

Each step prints a line like "-> initialize" and "<- evo-boundary 0.1.0"
so a person watching can see the handshake happen. Close the subprocess
cleanly. No third-party packages, no async.
```

```
python3 mcp_probe.py "where did I look for hiking clothes"
python3 mcp_probe.py "what is the capital of france"
```

> **WHY THIS IS THE BLOCK THAT WINS THE ROOM**
> Every other team shows a chat window. You show a protocol: an agent discovering a capability, calling it, and getting evidence or a refusal. It takes fifteen seconds and it says "this is infrastructure" louder than any slide could.

C3 · Only if there is time: the real client

If Claude Desktop is already installed and you have ten spare minutes, add the server to its config and ask it a question. It is a bonus, not the plan. The probe is the plan.

12:40 - 13:40

Block D - The voice, and the insurance

Two jobs. The voice is the track requirement and the emotional beat. The recording is what saves you if the projector, the wifi or the laptop decides to have a day.

D1 · Sarvam, in the smallest possible surface

The integration is already in evo_server.py as speak(). You are not building a voice assistant. You are changing the language the answer arrives in, which is exactly the right size of integration for a six-hour event.

```
export SARVAM_API_KEY=...          # from the track desk
python3 evo_server.py serve

# then in the UI press the language toggle, or:
curl -s "localhost:8765/ask?q=where+did+i+look+for+hiking+clothes&lang=hi-IN"
```

PASTE INTO KIRO · exports/ship-happens/evo_server.py

```
The function speak() calls Sarvam text-to-speech. Check it against the
current docs at docs.sarvam.ai and correct ONLY the request payload if
the field names have changed. Keep the function's signature, keep the
try/except that returns None, and keep the one-attempt limit.

Then add SARVAM_LANG, an environment variable that sets the default
target_language_code, so I can switch languages without editing code.
Print the effective language on startup next to the sarvam status line.
```

IF THE KEY NEVER ARRIVES

Do not fake it and do not apologise twice. Say once: "the voice path is built and gated behind a key that hasn't landed; everything else you're about to see runs locally." Then demo the text path, which is the substance anyway. A team that is straight about a missing key reads as more credible than one that mumbles through a broken feature.

D2 · The backup recording, which is not optional

Screen-record four minutes covering: the service starting, the works list, one good question, one refusal, the two break-it buttons, and the voice. Talk while you record. Do it at 13:15, not at 16:20 when you are out of time and out of patience.

```
# macOS, no installs
Cmd-Shift-5  ->  Record Selected Portion  ->  record  ->  save to Desktop
```

D3 · Freeze

13:40. Commit everything. From here you change words, not code. The single biggest cause of losing a hackathon is a change made after the last rehearsal.

PART SIX

Sprint 02: 14:30 to 16:00

Time	Do	Output
14:30 - 15:10	Speaking the question, not just hearing the answer (Speech-to-text via Sarvam, recorded in the browser).	A judge can talk to it.
15:10 - 15:25	Write the submission. Paste from the next page.	Submitted draft, not a blank form.
15:25 - 15:45	Hostile drill. Eight questions, out loud.	You stop being surprised by questions.
15:45 - 15:55	Rehearse twice against the clock.	You finish at 2:40, not 4:10.
15:55 - 16:00	Submit. Close the laptop lid on code.	Done.

Sprint 02 is where most teams lose, and it is not because they run out of time. It is because they keep building past the point where building helps. The marginal feature is worth far less than the marginal rehearsal.

The speech-to-text prompt, if you get there

PASTE INTO KIRO · evo_server.py + evo_ui.html

```
Add speech-to-text, using the browser rather than the terminal.

Server: a POST /stt endpoint accepting a raw audio body (webm or wav),
forwarding it to Sarvam's speech-to-text API with language_code hi-IN,
and returning {"text": "..."}. Standard library only. On any failure
return a 200 with {"text": "", "error": "..."} - never a 500, because a
demo must not show a stack trace.

UI: a microphone button next to the ask input that records while held,
posts the blob to /stt, writes the transcript into the input, and then
calls the existing ask(). Show a small red dot while recording.

Read the current Sarvam docs first and use their field names.
```

WHY THIS IS WORTH THE FORTY MINUTES

A judge pressing a microphone and asking a question in their own language, and getting an answer with evidence underneath it, is the moment they remember when they are deciding. Everything before it is demonstration; this is participation.

PART SEVEN

The submission, field by field

Write it at 15:10, not at 15:58. Paste and adjust.

Title

```
Evo — the boundary layer for AI agents
```

One line

```
Evo derives where one body of work ends and the next begins, on the
device, so agents can act on a real machine without a human labelling
every folder first.
```

Description

```
Every AI agent today is TOLD where its work begins and ends: you point it
at a folder. None of them DERIVE it. That is why agents still cannot
safely act on a real desktop - they do not know that this repository is
one job and that deck is another. Scale does not fix it, because the
boundary is not in the content. Two people can have identical files and
be doing different jobs.

Evo derives the boundary locally, as a side effect of a product people
use every morning, and exposes it for agents to query.

What you can try in the demo:
- The service: curl a work, its resume bundle, and a refusal.
- The protocol: an agent calls Evo over MCP and gets an answer with
  evidence and the works it rejected.
- The refusal: a boundary that guesses is not a boundary.
- The gate: declare two works the same and the build fails, on purpose.

Built today: the service, the MCP surface, and the voice path through
Sarvam. Built before today: the grouping engine - 16 Rust crates,
deterministic, no HTTP client. We say that plainly because the
architecture is the interesting part, not the timeline.

Track tool: Sarvam, for speech in and out. The model changes the
language; it does not decide the answer.
```

Stack

```
Rust (16 crates, the engine) · Python 3 standard library (service,
MCP, UI server) · Sarvam AI (speech in/out) · no cloud in the decision
path · no model decides the boundary
```

Links

```
Repo:   github.com/BHAVIT-EVO/architecture-v1
Demo:   exports/ship-happens/  (evo_server.py, evo_ui.html, mcp_probe.py)
Video:  your backup recording
```

> **SAY THE TIMELINE SENTENCE YOURSELF**
> "The engine is months old; today I built the layer that lets you talk to it." One sentence, at the top, in your own voice. It costs nothing and it removes the only way to lose the room - a judge discovering the repo is older than today and re-reading everything else you said.

PART EIGHT

Three minutes, in order

Fullscreen. One tab. Nothing else running. Times are targets, not rules.

	Say	Do
0:00	"Every knowledge worker has a second job nobody pays for: rebuilding what they were doing before the last interruption. It is not a search problem. The answer is not on the web. It is the state of your own machine."	Nothing. Let them look at you.
0:20	"This is my actual weekend. Ten thousand events, one machine, eight separate jobs - a codebase, a trek, a pitch, gear research, four more. I labelled them by hand. The engine independently claimed two of the eight and refused the other six."	Point at the works rail. The refused rows are grey; the claimed ones carry the accent.
0:40	"That gap is not a failure. A wrong merge is irreversible; a refusal costs nothing. It would rather say nothing than fuse a trek with a jacket - and every agent shipping today guesses instead."	Curl /works. Show the claimed count in the header.
1:00	"Ask it something."	Type the hiking-clothes question. Show the evidence line and the word rejected.
1:20	"And when it has no answer, it says so."	Ask about the capital of France.
1:30	"Now break it."	Merge by inference - refused. Then declare - the verifier goes red, exit 1.
2:00	"That is not a test suite. That is a verifier, and it is the part I would defend."	Hold the red panel on screen.
2:15	"Any agent can ask it, not just me."	Run mcp_probe.py. Show the handshake and the answer.
2:35	"Hindi."	Press the mic, ask, let the answer play.
2:50	"Every agent shipping right now has memory. None of them have jurisdiction. That is the layer."	Stop talking. Do not fill the silence.

> **THE FOUR WORDS THAT DECIDE IT**
> When the verifier goes red, resist explaining it. Say the sentence, hold the screen, and let the room work out what they just saw. Explaining steals the realisation, and the realisation is what they repeat to each other.

If you are cut to ninety seconds

Problem in one sentence. Ask one question. Break it once. The closing line. Drop the works tour and the voice. Never drop the refusal - it is the only part nobody else in the room has.

PART NINE

What they will ask

They ask	You say
"Did you build this today?"	"The engine is months old. Today I built the service, the MCP surface and the voice. I would rather say that than have you find it."
"Why would anyone need this?"	"Because their agents are useless on a real machine without it. You cannot hand-label a thousand folders per employee. The boundary has to be derived, once, passively."
"Isn't this just a folder?"	"A folder is declared in advance from a menu. This is derived from behaviour and carries its own evidence. Different artifact, different guarantees."
"How accurate is it?"	"164 resources I labelled by hand. It holds back 83% of its claims rather than asserting them. It refuses more than it says."
"Where's the AI?"	"Deliberately not in the decision. A boundary has to be deterministic and auditable. AI consumes it; there is an agent layer on the roadmap and I will not claim one exists."
"What about Cowork / Recall?"	"They carry context forward. This defines the boundary that context lives inside. A context engine with no jurisdiction is guessing."
"Business model?"	"Per seat, prosumer first. The wedge is people who lose an hour a day to this. I am not going to invent enterprise pricing for you."
"What's next?"	"Ten people outside the company running it for thirty days. Right now the boundary is proven on exactly one person."

> **THE ONE ANSWER THAT MUST NEVER LEAVE YOUR MOUTH**
> "We don't know, here's how we'd find out." is a strong answer. Every invented number is a permanent wound: one judge who knows the domain catches one invented number and everything else you claimed becomes suspect. You have real numbers. Use only those.

Drill it with a machine, not in your head

PASTE INTO CLAUDE / CHATGPT · before 15:45

```
You are a judge at an Indian student hackathon. You are technical, you
have seen forty AI projects today, and you are tired and sceptical. I have
three minutes.

Interrogate me one question at a time. After each answer, tell me in one
line what a sceptical judge would think of it, then ask the next question.
Escalate: what it does, then accuracy, then why it is not just a folder,
then why an incumbent will not simply build it, then the business.

Do not be encouraging. Stop after eight questions and tell me the two
weakest answers I gave.
```

PART TEN

When it breaks, and it will

Symptom	Cause	Do this, in the first minute
cargo fails to build boundary_dump	An API detail in the dump binary differs from what the engine exposes.	Delete threads.json. Restart the server; it falls back to boundary.json. Say "engine output is precomputed for portability" once.
The service starts but /works is empty	Neither threads.json nor boundary.json is where the server looks.	cd into exports/ship-happens and run it from there. The server resolves relative to its own file.
Sarvam returns 401	Key not exported in the shell that started the server.	export SARVAM_API_KEY then restart. If still failing, drop the voice and move on.
The mic does not record	Browser blocked it, or not on localhost.	Use http://localhost, never the file path. Grant permission on the first press.
MCP client hangs	The client waits for a response to a notification.	Never reply to notifications. Notifications have no id, by design.
Judge does not understand it	You explained architecture before the problem.	Stop. Say the problem sentence. Then say the boundary sentence. Then show the refusal.
Something crashes on stage	Any of the above, live.	Switch to the recording without comment and keep narrating over it.

The four sentences you must not lose

If the laptop dies, the wifi dies, and the projector will not talk to your machine, you can still give the pitch out loud. These four sentences are the pitch:

```
1.  Every agent today is TOLD where its work begins and ends.
2.  None of them DERIVE it, and content cannot give it to them.
3.  Evo derives it locally, and lets any agent ask.
4.  A boundary that guesses is not a boundary.
```

If you can say those four from memory, you cannot lose the room, only the prize.

APPENDIX

Everything you need in one place

Commands

```
# engine -> json
cargo run -p evo-threads --bin boundary_dump -- \
  --events tools/threads-regression/sample_events.json \
  --out exports/ship-happens/threads.json

# the service + UI
cd exports/ship-happens && python3 evo_server.py serve

# the protocol
python3 evo_server.py mcp
python3 mcp_probe.py "where did I look for hiking clothes"

# the offline explorer (no server, no install)
open boundary.html
```

Endpoints

```
GET  /                 the product UI
GET  /works            all works with resume points
GET  /works/{id}       one work with its restore set
GET  /ask?q=...        the boundary answer
GET  /ask?q=...&lang=hi-IN   same, spoken via Sarvam
GET  /health           source, work count, whether Sarvam is on
```

MCP tools

```
list_works   all bodies of work on this machine
find_work    which work a resource or phrase belongs to, with evidence
resume_work  the exact surfaces to reopen for a work
```

Numbers you may quote, and only these

```
10,392   recorded events in the corpus (13-15 September 2026)
   164   resources labelled by hand
     8   bodies of work I labelled by hand
     2   of those the engine claims on its own; the other six it refuses
     0   HTTP clients in the engine; a test fails the build if one appears
     1   person the boundary is proven on
```

Everything else - users, revenue, retention - is not yours to quote yet. Say "nobody outside the company has run it yet" and watch how much more they believe the rest.